计算机组成与结构
—运算方法1

李瑞　　教授
蒋志平　副教授
计算机科学与技术学院

温故 —关于数制表示

- 8位原码的表达范围？ -127， 0， 127的8位原码是?
- 8位反码的表达范围？ -127， 0， 127的8位反码是?
- 8位补码的表达范围？ -127， 0， 127的8位补码是?
- 8位移码的表达范围？ -127， 0， 127的8位移码是?
- 计算机中，如何表示实数?
- 浮点数的规格化是干嘛的？有嘛用?
- 0.001[补]如何规格化?
- IEEE 754格式？相比一般的规格化浮点，有什么特殊点?

温故 —关于数制表示

- 56的8421格式BCD码是?
- 简述ASCII码的特性?
- 简述Unicode的特性?
- 简单奇偶校验的用途和原理?
- “1110111”在奇/偶校验编码时，如何设置校验位?
- 简述海明码的用途和原理?
- 当海明码校验位输出为0101时，如何解读?
- 简述CRC的用途和工作原理?

本节学习要点

- 定点数（整数、纯小数）的四则运算
  - 加减法
- 算数逻辑部分
  - 单元电路
  - 算法逻辑单元ALU
  - 运算器的结构
- 定点数乘法
  - 原码乘法（一位/二位乘法）

定点数运算—加减法

定点数运算—加减法

- 无符号加法很简单，只需注意进位...

原码加法，6+7=13

	0 1 1 0	6
+	0 1 1 1	7
	1 1 0 1	13

原码加法，13+7=20

	1 1 0 1	13
+	0 1 1 1	7
	1 0 1 0 0	20

溢出

定点数运算—加减法

- 无符号加法很简单，只需注意进位...
- 减法比较麻烦（包括 加负数）

原码减法，13-7=6

0 1 1 0 1	13
- 0 0 1 1 1	7
0 0 1 1 0	6

借位机制麻烦

原码减法，-7+13=6

1 0 1 1 1	-7
+ 0 1 1 0 1	13
0 0 1 0 0	4

错误！

定点数运算—加减法

- 无符号加法很简单，只需注意进位...
- 减法比较麻烦（包括 加负数）
- 如何简化减法?
- 对减数预处理
  - 方案一：反码减法

原码减法，13-7=6

0 1 1 0 1	13
- 0 0 1 1 1	7
0 1 1 0	6

反码减法，13-7=6

0 1 1 0 1	13
+ 1 1 0 0 0	8
0 0 1 0 1	5
+ 1	1
0 0 1 1 0	6

7的反码

反码结果需+1

定点数运算—加减法

- 无符号加法很简单，只需注意进位...
- 减法比较麻烦（包括 加负数）
- 如何简化减法?
  - 对减数预处理
    - 方案一：反码减法
    - 方案二：补码减法

原码减法，13-7=6

0 1 1 0 1	13
- 0 0 1 1 1	7
0 1 1 0	6

反码减法，13-7=6

0 1 1 0 1	13
+ 1 1 0 0 0	8
0 0 1 0 1	5
+ 1	1
0 0 1 1 0	6

补码减法，13-7=6

0 1 1 0 1	13
+ 1 1 0 0 1	9
0 0 1 1 0	6

7的补码

定点数运算—加减法

- 无符号加法很简单，只需注意进位...
- 减法比较麻烦（包括 加负数）
- 如何简化减法?
  - 对减数预处理
    - 方案一：反码减法
    - 方案二：补码减法
- 补码加法统一加减法
  - [X + Y]补 = [X]补 + [Y]补
  - [X - Y]补 = [X]补 + [-Y]补

原码减法，13-7=6

0 1 1 0 1	13
- 0 0 1 1 1	7
0 1 1 0	6

反码减法，13-7=6

0 1 1 0 1	13
+ 1 1 0 0 0	8
0 0 1 0 1	5
+ 1	1
0 0 1 1 0	6

补码减法，13-7=6

0 1 1 0 1	13
+ 1 1 0 0 1	9
0 0 1 1 0	6

7的补码

定点数运算—加减法

- 补码加减法统一运算规则：
  - 都用补码表示
  - 减法变成加负数
  - 符号也参与运算
- 结果就是和/差的补码

- 例，用补码计算：
  - 63 + 35
  - -63 + (-35)
  - 63 - 35

- 解：

$[63]_{补}$ ＝00111111

$[-63]_{补}$ ＝11000001

$[35]_{补}$ ＝00100011

$[-35]_{补}$ ＝11011101

63 + 35

	0	0 1 1 1 1 1 1	63
+	0	0 1 0 0 0 1 1	35
	0	1 1 0 0 0 1 0	98

-63 + (-35)

	1	1 0 0 0 0 0 1	-63
+	1	1 0 1 1 1 0 1	-35
	1 1	0 0 1 1 1 1 0	-98

63 - 35

	0	0 1 1 1 1 1 1	63
+	1	1 0 1 1 1 0 1	-35
	1 0	0 0 1 1 1 0 0	28

这溢出的1怎么搞?

定点数运算—加减法

- 除了补码，移码也能加减
- 运算规则：
  - 都用移码表示
  - 减法变为加负数
  - 符号也参与计算
  - 运算结果符号位取反

结果就是和/差的移码

- 例，用移码计算：
  - 63 + 35
  - -63 + (-35)
  - 63 - 35

- 解：

$[63]_{移}$ ＝10111111

$[-63]_{移}$ ＝01000001

$[35]_{移}$ ＝10100011

$[-35]_{移}$ ＝01011101

63 + 35

1	0 1 1 1 1 1 1 1	63
+ 1	0 1 0 0 0 1 1	35
1 0	1 1 0 0 0 1 0	-30
+ 0 1	0 0 0 0 0 0 0	
1 1	1 1 0 0 0 1 0	98

-63 + (-35)

0	1 0 0 0 0 0 1	-63
+ 0	1 0 1 1 1 0 1	-35
1	0 0 1 1 1 1 0	30
+ 1	0 0 0 0 0 0 0	
1 0	0 0 1 1 1 1 0	-98

63 - 35

1	0 1 1 1 1 1 1	63
+ 0	1 0 1 1 1 0 1	-35
0	0 0 1 1 1 0 0	-100
+ 1	0 0 0 0 0 0 0	
1	0 0 1 1 1 0 0	28

定点数运算—加减法

- 溢出导致两个问题：
  - 数据部分是否越界?
  - 符号如何确定?
- 粗暴解决方案：
  - 扩大数值位数
- 实用解决方案：
  - 判断是否溢出

63 + 85

	0	0	1	1	1	1	1	1	63
+	0	1	0	1	0	1	0	1	85
	1	0	0	1	0	1	0	0	-101

63 + 67

	0	1	0	0	0	0	0	1	65
+	0	1	0	0	0	0	1	1	67
	1	0	0	0	0	1	0	0	-124

- 溢出判断方法：
  - 双符号位（变形补码）
  - 进位判别法
  - 符号位与进位标志判别法
  - ~~运算前后对比法~~

定点数运算—加减法

- 双符号位法
  - “一个符号不够，两个来凑”
  - 符号位S变成S_2S_1
    - 正数->00
    - 负数->11
- 溢出判断条件：$OF = S_1 \oplus S_2$

$$\begin{cases} S_1S_2 = 00, \textbf{Non-Overflow} \\ S_1S_2 = 00, \textbf{Non-Overflow} \\ S_1S_2 = 10, \textbf{Negative Overflow} \\ S_1S_2 = 01, \textbf{Positive Overflow} \end{cases}$$

双符号位

65 + 67			65 + 85		
0 0	1 0 0 0 0 0 1	65	0 0	0 1 1 1 1 1 1	63
+ 0 0	1 0 0 0 0 1 1	67	+ 0 0	1 0 1 0 1 0 1	85
0 1	0 0 0 0 1 0 0	溢出	0 1	0 0 1 0 1 0 0	溢出

正溢出

-65 + -67			-65 + -85		
1 1	0 1 1 1 1 1 1	-65	1 1	0 1 1 1 1 1 1	-65
+ 1 1	0 1 1 1 1 0 1	-67	+ 1 1	0 1 0 1 0 1 1	-85
1 0	0 1 1 1 1 0 0	溢出	1 0	1 1 0 1 0 1 0	溢出

负溢出

65 + 16			-65 - 5		
0 0	1 0 0 0 0 0 1	65	1 1	0 1 1 1 1 1 1	-65
+ 0 0	0 0 0 1 0 0 0	16	+ 1 1	1 1 1 1 0 1 1	-5
0 0	1 0 0 1 0 0 1	81	1 1	0 1 1 1 0 1 0	-70

无溢出

定点数运算—加减法

- 进位判别法
  - “其实就是双符号位法的变形”
  - 设C_{n-1}为最高数值位向符号位的进位，C_n是符号位向更高位的进位，则溢出判断条件：

$$OF = C_{n-1} \oplus C_n$$

$$\begin{cases} C_{n-1}C_n = 00, \textbf{Non-Overflow} \\ C_{n-1}C_n = 00, \textbf{Non-Overflow} \\ C_{n-1}C_n = 10, \textbf{Negative Overflow} \\ C_{n-1}C_n = 01, \textbf{Positive Overflow} \end{cases}$$

65 + 67

	0	1	0	0	0	0	0	1	65
+	0	1	0	0	0	0	1	1	67
	1	0	0	0	0	1	0	0	溢出

0 1

65 + 85

	0	0	1	1	1	1	1	1	63
+	0	1	0	1	0	1	0	1	85
	1	0	0	1	0	1	0	0	溢出

0 1

正溢出

向更高进位

-65 + -67

	1	0	1	1	1	1	1	1	-65
+	1	0	1	1	1	1	0	1	-67
	0	0	1	1	1	1	0	0	溢出

1 0

-65 + -85

	1	0	1	1	1	1	1	1	-65
+	1	0	1	0	1	0	1	1	-85
	0	1	1	0	1	0	1	0	溢出

1 0

负溢出

65 + 16

	0	1	0	0	0	0	0	1	65
+	0	0	0	1	0	0	0	0	16
	0	1	0	0	1	0	0	1	81

0 0

-65 - 5

	1	0	1	1	1	1	1	1	-65
+	1	1	1	1	1	0	1	1	-5
	1	0	1	1	1	0	1	0	-70

1 1

无溢出

定点数运算—加减法

- 符号位与进位标志判别法
  - CPU的处理器状态字（Processor State Word, PSW）描述众多运算状态，如溢出标志(Overflow Flag, OF)，进位标志(Carrier Flag, CF), 符号标志(Sign Flag)等…
- 当运算发生进位时，CF置1
- “把CF，SF纳入计算”
- 溢出判断条件：$OF = CF \oplus SF$

$$\begin{cases} CFSF = 00, \textbf{Non-Overflow} \\ CFSF = 00, \textbf{Non-Overflow} \\ CFSF = 10, \textbf{Negative Overflow} \\ CFSF = 01, \textbf{Positive Overflow} \end{cases}$$

CPU符号标志SF

CPU进位标志CF

正溢出

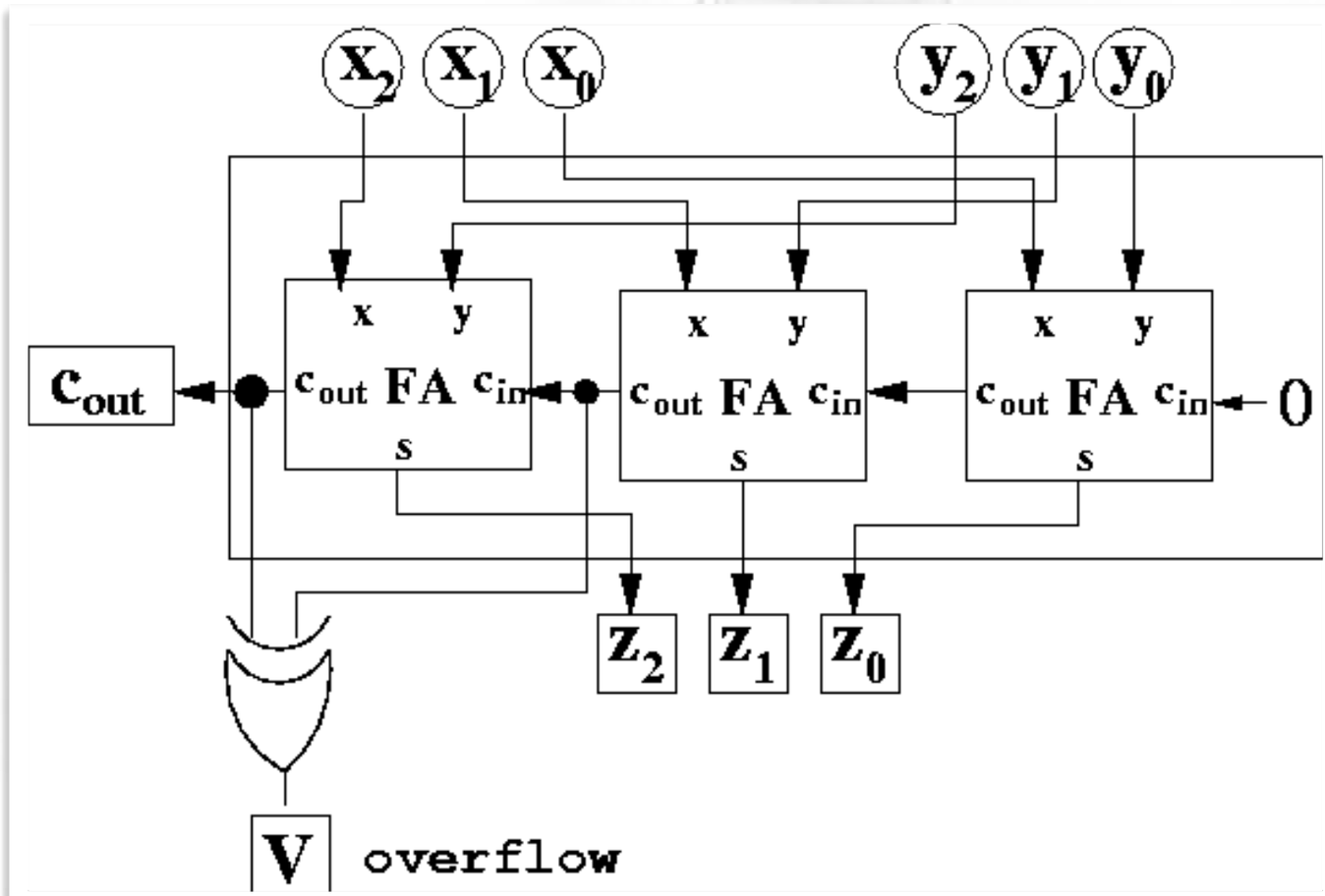
负溢出

无溢出

65 + 67			65 + 85		
0 1 0 0 0 0 0 1	65		0 0 1 1 1 1 1 1	63	
+ 0 1 0 0 0 0 1 1	67		+ 0 1 0 1 0 1 0 1	85	
0 1 0 0 0 0 1 0 0	溢出		0 1 0 0 1 0 1 0 0	溢出	
-65 + -67			**-65 + -85**		
1 0 1 1 1 1 1 1	-65		1 0 1 1 1 1 1 1	-65	
+ 1 0 1 1 1 1 0 1	-67		+ 1 0 1 0 1 0 1 1	-85	
1 0 0 1 1 1 1 0 0	溢出		1 0 1 1 0 1 0 1 0	溢出	
65 + 16			**-65 - 5**		
0 1 0 0 0 0 0 1	65		1 0 1 1 1 1 1 1	-65	
+ 0 0 0 1 0 0 0 0	16		+ 1 1 1 1 1 0 1 1	-5	
0 0 1 0 0 1 0 0 1	81		1 1 0 1 1 1 0 1 0	-70	

定点数运算—加减法

- 那么CPU实际是怎么做的?
  - SF和CF输入组合逻辑电路判断出来的~
- Then ?
  - 溢出标志OF = On
  - [maybe]产生溢出中断



3个全加器级联

加法器逻辑电路

加法器逻辑电路

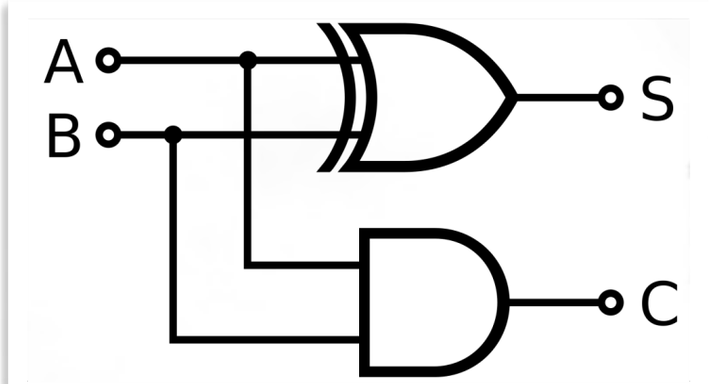
- 加法器
  - 一位加法器
    - 半加器(Half Adder)
    - 全加器(Full Adder)
  - 多位加法器
    - 行波加法器（RCA）
    - 先行进位加法器（CLA）
- BCD加法器

加法器逻辑电路

- 半加器(Half Adder)
  - 没有进位输入(Carry In)的加法器
  - S_i为和，C_{i+1}为进位
  - 电路逻辑为：

$$S_i = A \oplus B$$

$$C_{i+1} = A \cdot B$$



一位半加器

Inputs		Outputs	
A	B	C	S
0	0	0	0
1	0	0	1
0	1	0	1
1	1	1	0

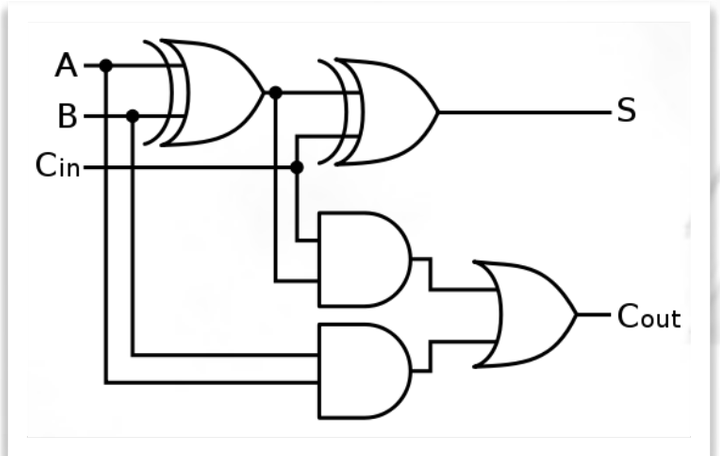
半加器真值表

加法器逻辑电路

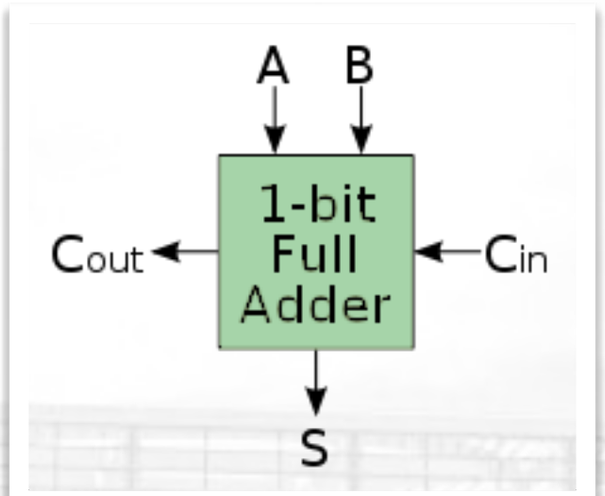
- 全加器(Full Adder)
  - 包含进位输入(Carry In)的加法器
  - 电路逻辑为：

$$S_i = A \oplus B \oplus C$$

$$C_{i+1} = A \cdot B + (A \oplus B) \cdot C_{in}$$



一位全加器



一位全加器

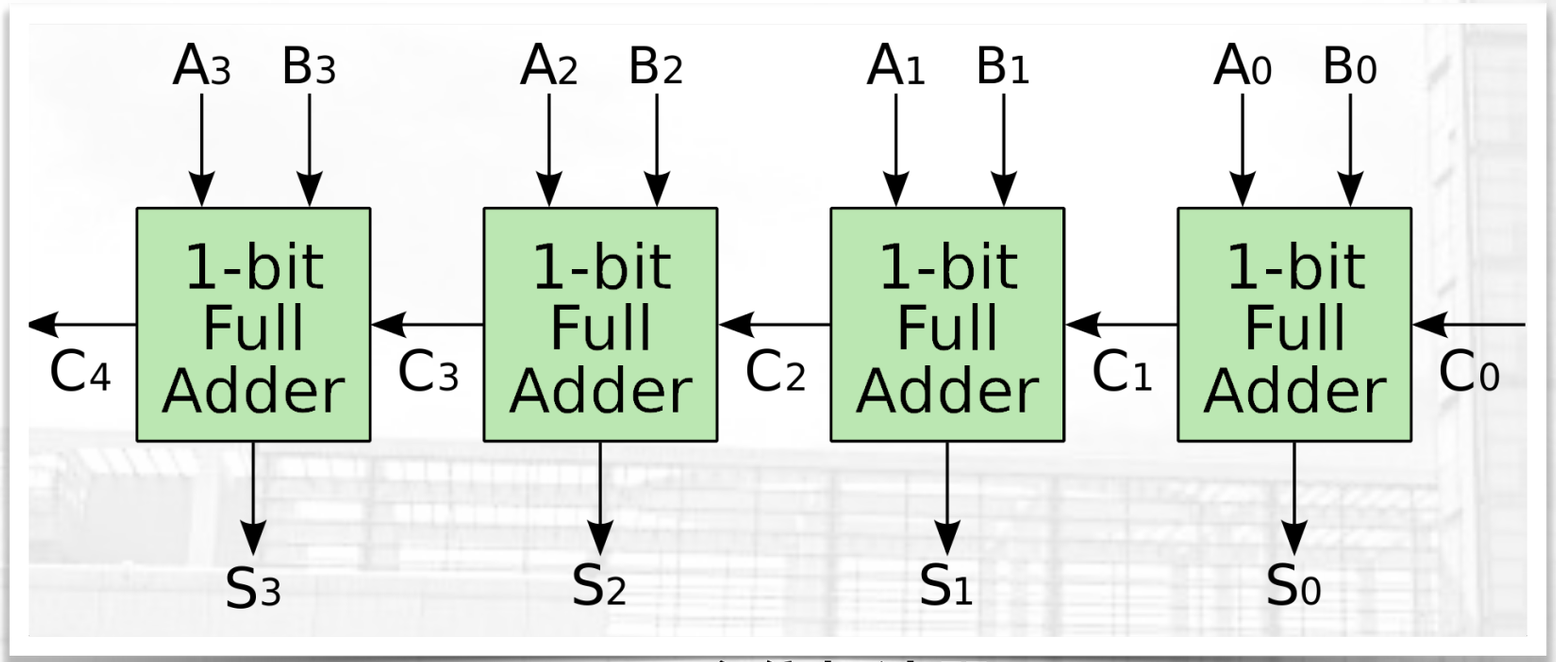
Inputs			Outputs	
A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

全加器真值表

加法器逻辑电路

行波多位加法器(Ripple-Carry Adder, RCA)

- **自低位向高位，依次计算、进位，计算、进位…**
  - **so-called “ripple carry”**
- **RCA结构简单，但有明显缺点：计算时延大，$n\Delta t$**



RCA多位加法器

加法器逻辑电路

先行进位加法器(Carry-Lookahead Adder, CLA)

- 回顾一下1位全加器的进位逻辑C_{i+1}：

$$C_{i+1} = A \cdot B + (A \oplus B) \cdot C_{in}$$

$$\text{let} \begin{cases} P_i \equiv A \oplus B \\ G_i \equiv A \cdot B \end{cases}, \Rightarrow C_{i+1} = G_i + P_i \cdot C_i$$

$$\begin{cases} C_1 = G_0 + P_0 C_{in} \\ C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{in} \\ C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in} \\ C_4 = G_3 P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in} \end{cases}$$

C_i进位根本不care C_{i-1}!

所有进位C可以根据P序列和G序列直接求得

加法器逻辑电路

先行进位加法器(Carry-Lookahead Adder, CLA)

- 回顾一下1位全加器的进位逻辑C_{i+1}：

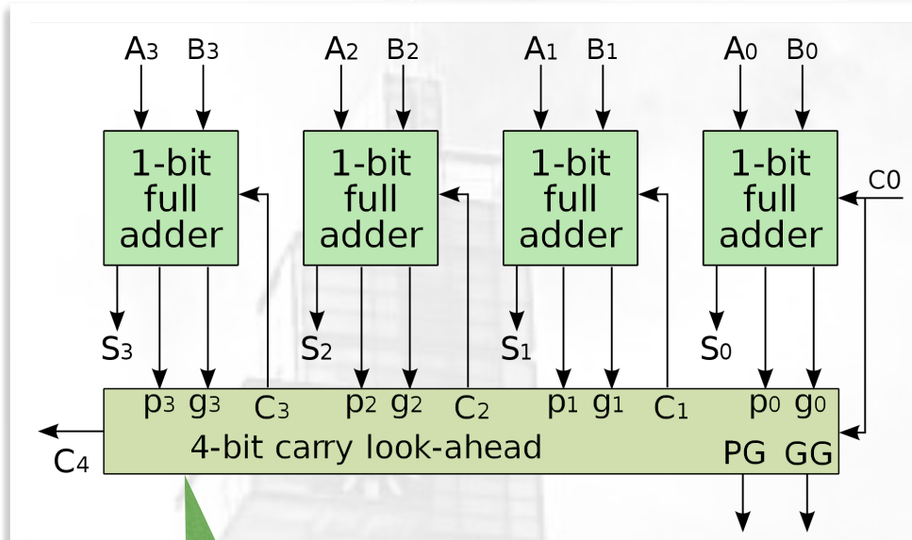
$$C_{i+1} = A \cdot B + (A \oplus B) \cdot C_{in}$$

$$\text{let} \begin{cases} P_i \equiv A \oplus B \\ G_i \equiv A \cdot B \end{cases}, \Rightarrow C_{i+1} = G_i + P_i \cdot C_i$$

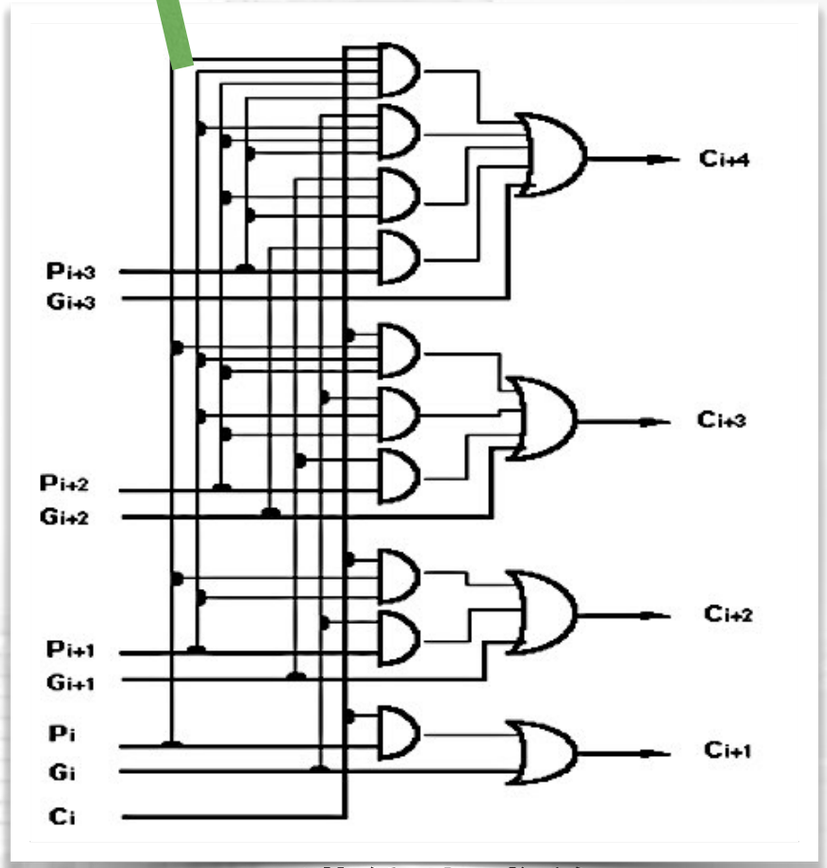
C_i进位根本不care C_{i-1}!

$$\begin{cases} C_1 = G_0 + P_0 C_{in} \\ C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{in} \\ C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in} \\ C_4 = G_3 P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in} \end{cases}$$

所有进位C可以根据P序列和G序列直接求得



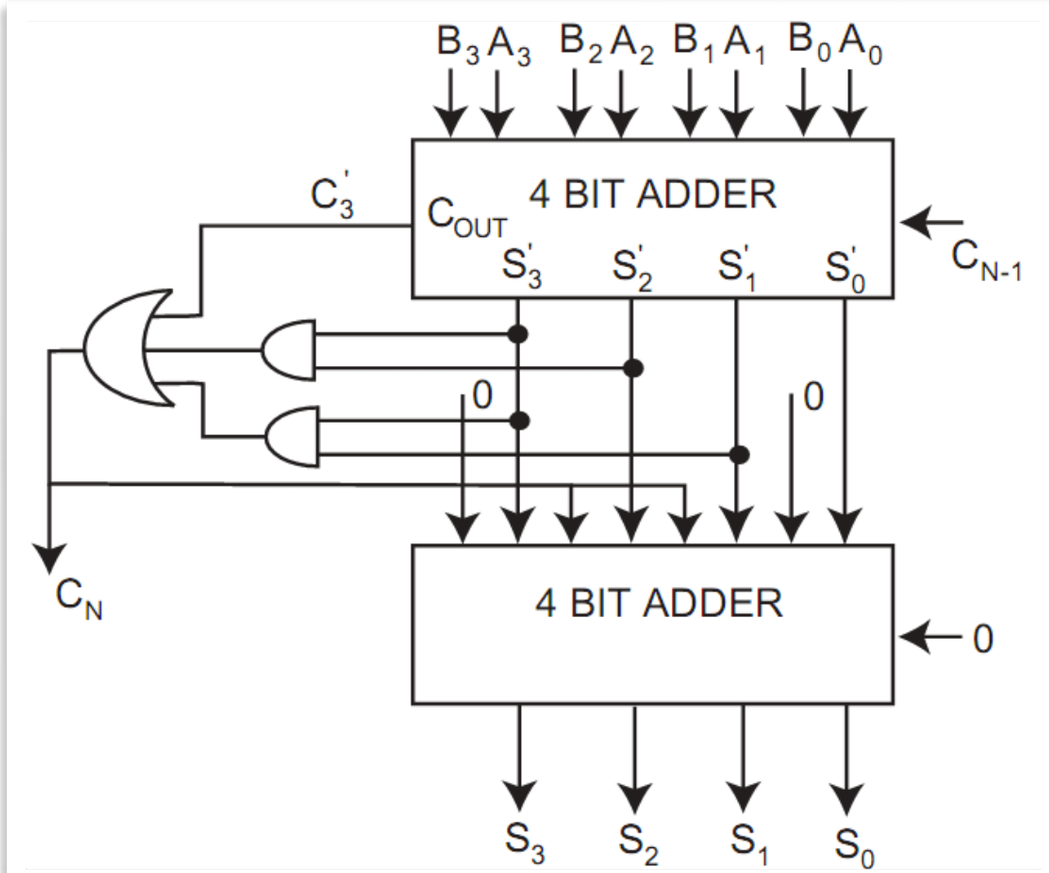
CLA 4位加法器



4bit 进位生成单元

加法器逻辑电路

- **BCD加法器**
  - **两个4bit加法器构成**
    - **一个加法器算数值**
    - **另一个加法器用来+06H ….**



4bit BCD加法器

加法器逻辑电路

现代的CPU用啥加法器?

- 五花八门...
  - 精确加法器
    - CLA / RCA
    - Kogge-Stone / Brent-Kung / Han-Carlson (CLA的高级形态)
  - 非精确加法器 Speculative Adder
    - Insight: “很少有很长的进位链，所以赌某段无进位，...”
    - 95%正确，比最快的精确加速器快平均1.5x
    - 有额外电路快速检查结果是否正确（只能判断正确）
- 取舍因素？ 设计过程 / 性能 / 面积 / 延时 / 功耗 ....
- 区区一个加法器，直到现在，还在不断地发展...

定点数运算—乘法

定点数运算—乘法

原码一位乘法就是模拟手工乘法:

- **符号**$S_m = S_A \oplus S_b$**, 数值**$|m| = |A| \times |B|$

例，[X]原=0.1101, [Y]原=1.1011，求积

解：符号，$S = 0 \oplus 1 = 1$**；数值部分如下：**

模拟手工乘法

```
          1 1 0 1
    ×     1 0 1 1
  ---------------
          1 1 0 1
        1 1 0 1
      0 0 0 0
    1 1 0 1
  ---------------
  1 0 0 0 1 1 1 1
```

如何描述这一过程?

定点数运算—乘法

原码一位乘法就是模拟手工乘法:

- **符号**$S_m = S_A \oplus S_b$**, 数值**$|m| = |A| \times |B|$

例，[X]原=0.1101, [Y]原=1.1011，求积

解：符号，$S = 0 \oplus 1 = 1$**；数值部分如下：**

模拟手工乘法

```
          1 1 0 1
    ×     1 0 1 1
    ---------------
          1 1 0 1
        1 1 0 1
      0 0 0 0
    1 1 0 1
    ---------------
  1 0 0 0 1 1 1 1
```

如何描述这一过程?

1. **设一临时变量**$D = 0$
2. **若**Y**末位为1，**$D = D + X$
3. D**右移一位；**Y**右移一位**
4. **若**Y**有剩余位数，回到第2步**
5. **结束**

定点数运算—乘法

原码一位乘法就是模拟手工乘法:

- 符号$S_m = S_A \oplus S_b$, 数值$|m| = |A| \times |B|$

例，[X]原=0.1101, [Y]原=1.1011，求积

解： 符号，$S = 0 \oplus 1 = 1$； 数值部分如下：

模拟手工乘法

```
          1 1 0 1
    ×     1 0 1 1
    ---------------
          1 1 0 1
        1 1 0 1
      0 0 0 0
    1 1 0 1
    ---------------
    1 0 0 0 1 1 1 1
```

1. 设一临时变量$D = 0$
2. 若Y末位为1，$D = D + X$
3. D右移一位；Y右移一位
4. 若Y有剩余位数，回到第2步
5. 结束

	D				A			A_0	操作
0	0	0	0	0	1	0	1	1	A_0=1, +X
+0	1	1	0	1					
0	1	1	0	1					
0	0	1	1	0	1	1	0	1	→右移一次
+0	1	1	0	1					A_0=1, +X
1	0	0	1	1	1	1	0	1	
0	1	0	0	1	1	1	1	0	→右移一次
0	0	0	0	0					A_0=0, +0
0	1	0	0	1	1	1	1	0	
0	0	1	0	0	1	1	1	1	→右移一次
+0	1	1	0	1					A_0=1, +X
1	0	0	0	1	1	1	1	1	
0	1	0	0	0	1	1	1	1	→右移一次

原码一位乘法运算过程

最终结果，[X]原x[Y]原=1.10001111

定点数运算—乘法

练习，[X]原=0.100111, [Y]原=1.100111，求积

1. 设一临时变量$D=0$
2. 若Y末位为1，$D=D+X$
3. D右移一位；Y右移一位
4. 若Y有剩余位数，回到第2步
5. 结束

定点数运算—乘法

练习，[X]原=0.101011, [Y]原=1.100101，求积

1. 设一临时变量$D=0$
2. 若Y末位为1，$D=D+X$
3. D右移一位；Y右移一位
4. 若Y有剩余位数，回到第2步
5. 结束

定点数运算—乘法

原码二位乘法，

一次乘两位数字？这个在十进制下很反人类啊...，但在二进制下，似乎还好...

```
  1 1 0 1
×   1 0 0 1
-----------
  1 1 0 1      +|X|
1 1 0 1 0      +2|X|
-----------
0 1 1 1 1 0 1 0 1
```

```
  1 1 0 1
×   1 1 0 0
-----------
  0 0 0 0      +0
1 0 1 0 1 1    +3|X|
-----------
1 0 1 0 1 1 0 0
```

规则似乎很直观：

$$
\begin{cases}
Y_{i+1}Yi = 00, +0 \\
Y_{i+1}Yi = 01, +|X| \\
Y_{i+1}Yi = 10, +2|X| \\
Y_{i+1}Yi = 11, +3|X|
\end{cases}
$$

定点数运算—乘法

原码二位乘法，

一次乘两位数字？这个在十进制下很反人类啊...，但在二进制下，似乎还好...

```
  1 1 0 1
×   1 0 0 1
-----------
  1 1 0 1      +|X|
1 1 0 1 0      +2|X|
-----------
0 1 1 1 0 1 0 1
```

```
  1 1 0 1
×   1 1 0 0
-----------
  0 0 0 0      +0
1 0 1 0 1 1    +3|X|
-----------
1 0 1 0 1 1 0 0
```

规则似乎很直观：

$$\begin{cases} Y_{i+1}Yi = 00, +0 \\ Y_{i+1}Yi = 01, +|X| \\ Y_{i+1}Yi = 10, +2|X| \\ Y_{i+1}Yi = 11, +3|X| \end{cases}$$

规则还可以更进一步优化：

$$+3|X| = +4|X| - |X|$$
$$= 2 \times +2|X| - |X|$$

⇒先减$|X|$，平移2步后$+|X|$

```
    0 0 1 1 0 1
×   0 0 1 1 0 0
-----------------
    0 0 0 0 0 0
  1 1 0 0 1 1    −|X|
0 1 1 1 1 0      +2|X|
-----------------
1 0 1 0 1 1 0 0
```

定点数运算—乘法

原码二位乘法

规则还可以更进一步优化：

$$+3|X| = +4|X| - |X|$$
$$= 2 \times +2|X| - |X|$$

⇒**先减$|X|$，平移2步后+$|X|$**

⇒

标记是否要$+4|X|$

Y_{i+1}	Y_i	C	操　作
0	0	0	+0，　右移2次，C＝0
0	0	1	+|X|，　右移2次，C＝0
0	1	0	+|X|，　右移2次，C＝0
0	1	1	+2|X|，　右移2次，C＝0
1	0	0	+2|X|，　右移2次，C＝0
1	0	1	－|X|，　右移2次，C＝1
1	1	0	－|X|，　右移2次，C＝1
1	1	1	+0，　右移2次，C＝1

定点数运算—乘法

原码二位乘法

标记是否要+4|X|

Y_{i+1}	Y_i	C	操　作
0	0	0	+0，右移2次，C＝0
0	0	1	+|X|，右移2次，C＝0
0	1	0	+|X|，右移2次，C＝0
0	1	1	+2|X|，右移2次，C＝0
1	0	0	+2|X|，右移2次，C＝0
1	0	1	－|X|，右移2次，C＝1
1	1	0	－|X|，右移2次，C＝1
1	1	1	+0，右移2次，C＝1

例，[X]原=0.1001111, [Y]原=1.100111，求积

解：[2X]补=01.001110; [-X]补=1.011001

符号位	D	A	操作
0 0 0 1 1 1	0 0 0 0 0 0 0 1 1 0 0 1	1 0 0 1 1 1	C=0 −X
1 1 1 1 1 1 0 0 1	0 1 1 0 0 1 1 1 0 1 1 0 0 0 1 1 1 0	0 1 1 0 0 1	C=1 →右移二次 C=1,+2X
0 0 1 0 0 0 0 0 1	0 0 0 1 0 0 0 1 0 0 0 1 0 0 1 1 1 0	0 0 0 1 1 0	C=0 →右移二次 C=0,+2X
0 0 1 0 0 0	0 1 1 1 1 1 0 1 0 1 1 1	1 1 0 0 0 1	C=0 →右移二次

图 3-9　例 3-12 的二位乘法的运算过程

原码两位乘法运算过程

最终结果，[X]原x[Y]原=1.010111110001

定点数运算—乘法

练习，[X]原=0.110110, [Y]原=1.101101，求积

解：[2X]补=01.101100; [-X]补=1.001010

定点数运算—乘法

练习，[X]原=0.101011, [Y]原=1.110111，求积

解：[2X]补=01.010110; [-X]补=1.010101

定点数运算—补码乘法

定点数运算—补码乘法

补码乘法

- ~~校正法~~
- Booth法
- ~~两位Booth法~~

定点数运算—补码乘法

```
          0 0 1 1 0 1
×         0 0 1 1 1 0
-----------------------
          0 0 0 0 0 0
        0 0 1 1 0 1
      0 0 1 1 0 1
    0 0 1 1 0 1
  0 0 0 0 0 0
0 0 0 0 0 0
-----------------------
0 0 0 1 0 1 1 0 1 1 0
```

<—无脑计算需要6步

v.s

略用心机只需3步—>

```
          0 0 1 1 0 1
×         0 0 1 1 1 0
-----------------------
        0 0 0 1 1 0 1
      0 0 1 1 0 1
    0 0 1 1 0 1
-----------------------
0 0 0 1 0 1 1 0 1 1 0
```

定点数运算—补码乘法

```
          0 0 1 1 0 1
×         0 0 1 1 1 0
-------------------------
          0 0 0 0 0 0
        0 0 1 1 0 1
      0 0 1 1 0 1
    0 0 1 1 0 1
  0 0 0 0 0 0
-------------------------
0 0 0 1 0 1 1 0 1 1 0
```

<—无脑计算需要6步

v.s

略用心机只需3步—>

能不能更简化！？

```
          0 0 1 1 0 1
×         0 0 1 1 1 0
-------------------------
        0 0 0 1 1 0 1
      0 0 1 1 0 1
    0 0 1 1 0 1
-------------------------
0 0 0 1 0 1 1 0 1 1 0
```

定点数运算—补码乘法

```
          0 0 1 1 0 1
×         0 0 1 1 1 0
-----------------------
          0 0 0 0 0 0
        0 0 1 1 0 1
      0 0 1 1 0 1
    0 0 1 1 0 1
  0 0 0 0 0 0
-----------------------
0 0 0 1 0 1 1 0 1 1 0
```

<—无脑计算需要6步

v.s

略用心机只需3步—>

能不能更简化！？

先看一个日常使用的技巧：

$$12345 \times 1001 = 12345 \times (1000 + 1)$$

$$2342 \times 999 = 2342 \times (1000 - 1)$$

对二进制，也一样！

$$1010 \times 0111 = 1010 \times (1000 - 1)$$

$$1010 \times 1110 = 1010 \times (10000 - 10)$$

```
          0 0 1 1 0 1
×         0 0 1 1 1 0
-----------------------
        0 0 0 1 1 0 1
      0 0 1 1 0 1
    0 0 1 1 0 1
-----------------------
0 0 0 1 0 1 1 0 1 1 0
```

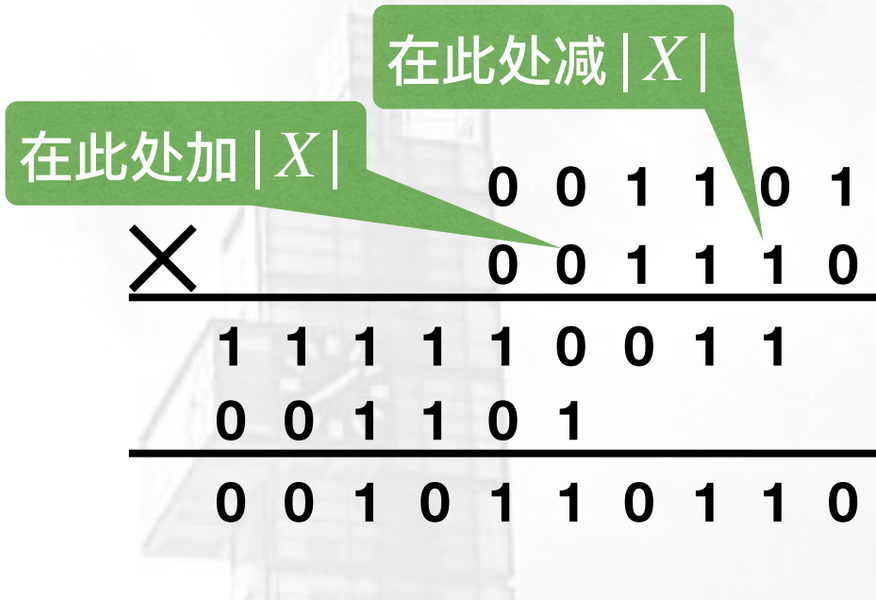
定点数运算—补码乘法

```
              0 0 1 1 0 1
        ×     0 0 1 1 1 0
    ---------------------
        0 0 0 1 1 0 1
        0 0 1 1 0 1
      0 0 1 1 0 1
    ---------------------
  0 0 0 1 0 1 1 0 1 1 0
```

<—略用心机需要3步

v.s

运用技巧只需2步—>



问题来了：如何高效地识别“一串1”的开始和结束?

定点数运算—补码乘法

```
              0 0 1 1 0 1
        ×     0 0 1 1 1 0
    ---------------------
        0 0 0 1 1 0 1
        0 0 1 1 0 1
      0 0 1 1 0 1
    ---------------------
  0 0 0 1 0 1 1 0 1 1 0
```

<—略用心机需要3步

v.s

运用技巧只需2步—>

在此处减 $|X|$

在此处加 $|X|$

```
              0 0 1 1 0 1
        ×     0 0 1 1 1 0
    ---------------------
    1 1 1 1 1 0 0 1 1
    0 0 1 1 0 1
    ---------------------
  0 0 1 0 1 1 0 1 1 0
```

问题来了：如何高效地识别“一串1”的开始和结束?

Booth算法：

- 从右向左，遇到“1 0”就是“一串1”的开始，在1的位置减 $|X|$；
- 从右向左，遇到“0 1”就是“一串1”的结束，在0的位置加 $|X|$；
- 中间的“00”和“11”都不管！

y_i	y_{i-1}	操 作
0	0	部分积＋0，右移1位
0	1	部分积＋$[X]_{补}$，右移1
1	0	部分积＋$[-X]_{补}$，右移1
1	1	部分积＋0，右移1位

定点数运算—补码乘法

Booth算法：

- 从右向左，遇到“1 0”就是“一串1”的开始，在1的位置减$|X|$；
- 从右向左，遇到“0 1”就是“一串1”的结束，在0的位置加$|X|$；
- 中间的“00”和“11”都不管！

【例】X＝0.1010，Y＝－0.1101，利用Booth法求积。

【解】：

$[X]_{补}$ ＝00.1010

$[-X]_{补}$ ＝11.0110

$[Y]_{补}$ ＝11.0011

在此处减$|X|$

符号	D	A	A_{-1}	操作说明
0 0	0 0 0 0	1 0 0 1 1	0	
1 1	0 1 1 0			$+[-X]_{补}$
1 1	0 1 1 0			
1 1	1 0 1 1	0 1 0 0 1	1	右移1位
0 0	0 0 0 0			+0
1 1	1 0 1 1			
1 1	1 1 0 1	1 0 1 0 0	1	右移1位
0 0	1 0 1 0			$+[X]_{补}$
0 0	0 1 1 1			
0 0	0 0 1 1	1 1 0 1 0	0	右移1位
0 0	0 0 0 0			+0
0 0	0 0 1 1			
0 0	0 0 0 1	1 1 1 0 1	0	右移1位
1 1	0 1 1 0			$+[-X]_{补}$
1 1	0 1 1 1	1 1 1 0 1	0	不移位

在此处加$|X|$

∴ **$[X \cdot Y]_{补}$ ＝1.01111110**

定点数运算—乘法

练习，[X]原=0.110110, [Y]原=1.101101，求积

解：[2X]补=01.101100; [-X]补=1.001010

定点数运算—乘法

练习，[X]原=0.101011, [Y]原=1.110111，求积

解：[2X]补=01.010110; [-X]补=1.010101